

PRODUCT REQUIREMENTS DOCUMENT

Yiyara: Connected Execution Platform

Planning engine, goal graph, accountability, teams, calendar scheduling, and momentum analytics

Status: Draft v0.1

Date: 10 August 2026

Product stage: Core goal decomposition and iterative plan console

Decision: Build trustworthy execution foundations before graph, social, or analytics UI

Recommendation: Treat these features as one platform direction, not separate add-ons. Canonical tasks, activity events, and workspace permissions form shared foundation. Extract planning engine through versioned contracts and shadow evaluation; avoid immediate rewrite-and-switch.

Where Yiyara should move next

Start with a two-week Planning Foundation Sprint. Do not begin with graph UI or full Go rewrite. First make domain and quality contracts explicit, measure current Gemini output, then build against evidence.

- Approve canonical Task, GoalTask, ActivityEvent, ContextSnapshot, DecompositionRequest, and DecompositionResult schemas.
- Create 100-200 representative goal prompts and score current output for actionability, latency, edits, and cost.
- Write service-boundary and migration architecture decisions, including Go acceptance gate.
- Define deterministic actionability rules and clarification behavior.
- Plan personal Workspace backfill and single-goal Task migration before feature code.

Contents

1. Executive decision
2. Current product state
3. Vision, users, goals, and principles
4. Target architecture
5. Decomposition engine
6. Domain model and migration
7. Goal connection graph
8. Accountability and teams
9. Calendar scheduling
10. Momentum analytics
11. API and event surface
12. Security and reliability
13. Delivery plan
14. Acceptance criteria and success metrics
15. Risks and decisions
16. Appendices

1. Executive decision

Yiyara should evolve from goal generator into connected execution system. User should receive tasks that can be scheduled, completed, verified, reused across goals, and explained through dependencies and momentum.

Separate orchestration engine is sound. Language split alone does not create model independence. Independence comes from canonical request and result schemas, provider adapters, deterministic validation, evaluation suites, and Django-owned persistence.

Build order: Canonical task model and events; reliable decomposition engine; calendar-ready scheduling; connected graph; accountability and team permissions; momentum analytics.

2. Current product state

Snapshot reflects current backend and frontend working trees, including uncommitted plan-console work.

Area	Current state	Primary gap
Authentication	Django users, internal auth bridge, JWT	No workspace or shared-resource authorization
Planning	Plan, goals, tasks persisted atomically	Tasks lack outcome, effort, scheduling constraints
Console	One conversation per plan; goal or board scope	No execution state workflow
AI changes	AI proposes; user applies or rejects; version conflicts	Gemini-specific path; no quality-repair pipeline
Versioning	Plan versions and immutable revisions	New domains must join same revision/event model
Guest flow	Anonymous preview and authenticated import	No persistent run or quality report
Tasks	One Goal foreign key; due date; completion boolean	No shared task, verifier, blockers, duration, history
Graph	Not implemented	No shared nodes, dependencies, or evidence
Teams	Not implemented	No workspace, membership, roles, assignment
Accountability	Not implemented	No verifier or completion review
Analytics	Completion counts	No events, pace, baselines, blockers
Calendar	Not implemented	No availability, schedule blocks, sync

Verified baseline: backend test suite passes 17 tests; frontend ESLint passes. Documentation needs correction because JWT is implemented while some roadmap text calls it planned, and OpenAI is described as supported while its provider file remains a placeholder.

3. Product vision

Yiyara converts ambitions into connected, scheduleable, socially supported execution plans.

- What must happen next?
- When can it happen?
- What outcome proves completion?
- Which goals benefit?
- What blocks it?
- Who verifies it?

- Is momentum rising or falling?
- Why does Yiyara believe this?

4. Product goals and non-goals

Primary goals

- Produce calendar-ready tasks, not vague planning language.
- Remain model agnostic without weakening validation.
- Reuse one task across several goals.
- Expose task and goal dependencies.
- Support mutual accountability and manager-verified team work.
- Show explainable momentum and stalled goals.
- Keep Django authoritative for data, permissions, and accepted changes.
- Preserve user approval before AI or calendar mutations.

Non-goals

- Fully autonomous calendar changes.
- AI deciding whether human work was genuinely completed.
- Public social network or employee ranking system.
- Full Jira or Asana replacement.
- Automatic cross-user task merging.
- Graph database during initial release.

5. Users and roles

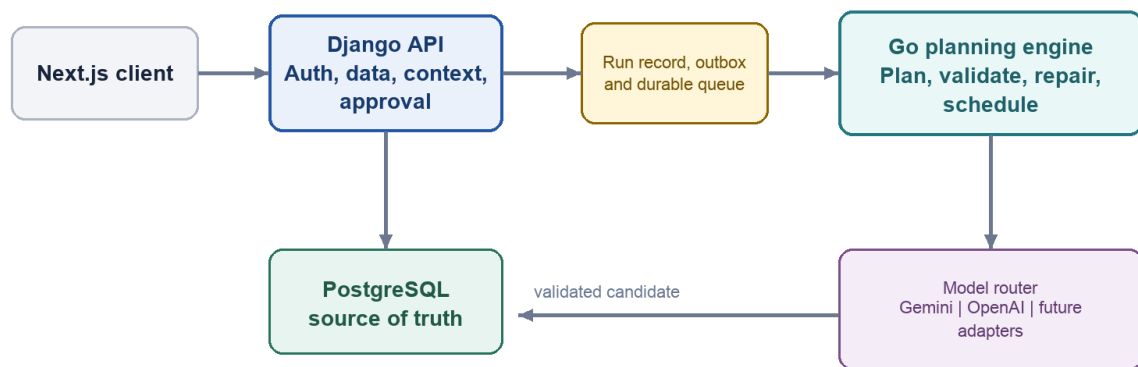
Persona	Core need
Solo user	Decomposes ambitions, schedules actions, tracks momentum.
Accountability partner	Verifies selected tasks without seeing unrelated data.
Team member	Performs assigned work and requests completion.
Manager	Creates team goals, assigns work, verifies under configured policy.
Workspace owner	Controls membership and workspace settings.

6. Product principles

- AI proposes; domain layer validates; authorized human accepts.
- One task means one real unit of work.
- Shared task stored once and linked many times.
- Completion uses state transitions, not mutable boolean.
- Connections carry evidence and provenance.
- Personal data remains private unless explicitly shared.
- Metrics explain themselves.
- Context sent to models stays minimal and consent-filtered.
- Every mutation is idempotent and auditable.

7. Target architecture

Target architecture and ownership boundary



Rule: engine receives a versioned, consent-filtered context snapshot. It never receives Django DB credentials or calendar OAuth tokens.

Django ownership

- Authentication, authorization, users, workspaces, and memberships
- Plans, goals, tasks, graph edges, revisions, and audits
- Calendar credentials and external writes
- Context assembly, consent filtering, and run records

- Final validation, persistence, accountability, analytics, and notifications

Engine ownership

- Prompt and schema versions
- Planning stages and provider routing
- Actionability scoring and targeted repair
- Dependency, duplicate, and schedule suggestions
- Token, cost, latency, and attempt metadata

Engine receives immutable context snapshot. It receives no Django database credentials, user JWT, or calendar OAuth token. It returns candidates, never accepted domain mutations.

Transport evolution

1. Initial extraction: internal HTTP, persisted Django run, strict timeout, idempotency, synchronous preview compatibility.
2. Scale stage: transactional outbox, durable queue, at-least-once delivery, idempotent consumers, polling or Server-Sent Events.
3. Result stage: engine publishes candidate; Django validates again and persists proposal or clarification.

8. Decomposition engine

Run contract

DecompositionRequestV1 contains request and idempotency identifiers, operation, schema version, ambition, timezone, constraints, availability summary, existing plan snapshot, permitted context, provider policy, and source versions.

DecompositionResultV1 contains result status, candidate goals, canonical tasks, local dependency references, schedule proposals, relationship suggestions, clarification questions, quality report, version metadata, usage, cost, latency, and attempt counts.

Run states

- queued
- running
- needs_clarification
- candidate_ready

- accepted
- rejected
- failed
- cancelled

Provider adapter

- Generate schema-bound result and normalize output.
- Report usage and provider request identifier.
- Classify retryable and non-retryable errors.
- Expose structured-output capabilities.
- Support timeout and cancellation.
- Normalize safety refusals without provider leakage into domain code.

Provider-specific prompt compilation stays inside adapter. Domain code consumes canonical structs. Go is viable because official OpenAI and Google Go SDKs support required API patterns; language decision remains gated by a short conformance spike.

Quality pipeline

1. Validate request and context freshness.
2. Generate candidate plan.
3. Validate JSON schema.
4. Validate dates, ownership, limits, and references.
5. Score every task for actionability.
6. Repair only failed goals or tasks.
7. Detect duplicates and dependency cycles.
8. Test schedule fit against availability.
9. Return candidate or clarification.
10. Django validates again before persistence.

Calendar-ready task standard

- Explicit action and observable outcome
- Completion criteria

- Estimated effort
- Deadline or scheduling window
- Required context or location
- Dependencies
- Splittable or non-splittable flag
- Minimum useful work block
- Confidence and assumptions

Quality example: 'Work on portfolio' fails. 'Draft introduction and outcomes for case study one; done when 500-800 words exist in document' passes.

Task and calendar block remain separate. A four-hour task may remain one task but receive several 60-minute schedule blocks.

Retry policy

Path	Trigger	Bound
Network retry	Timeout, 408, 429, 5xx only	Maximum 3; exponential backoff and jitter
Schema repair	Malformed structured output	Maximum 2 targeted attempts
Quality repair	Failed actionability nodes	Maximum 2; repair failed nodes only
Provider fallback	Provider failure or capability mismatch	One attempt when policy allows
Clarification	Missing or ambiguous user constraints	Return questions; do not invent detail

Every run has elapsed-time, call, token, and cost ceilings. Authentication and malformed request failures never retry.

9. Domain model and migration

Entity	Purpose
Workspace	Personal or team security boundary
Membership	User role and membership state
Plan	Workspace-owned ambition board with version
Goal	Outcome inside plan
Task	Canonical work item, independent from one goal

Entity	Purpose
GoalTask	Many-to-many link, order, contribution weight, required flag
TaskDependency	Directed blocking or enabling relationship
GoalRelationship	Supports, overlaps, conflicts, duplicates
RelationshipSuggestion	AI suggestion, evidence, confidence, review state
TaskAssignment	Performer and verifier assignments
CompletionReview	Request, approval, rejection, evidence, reviewer
ActivityEvent	Immutable execution and mutation ledger
DecompositionRun	Run state, context version, accepted candidate
DecompositionAttempt	Provider call, repair reason, latency, cost
ScheduleBlock	Proposed or synced calendar allocation
CalendarConnection	Encrypted integration metadata
MomentumSnapshot	Versioned metric values

Task state machine

todo -> in_progress -> pending_verification -> done. Work may move into blocked, return to in_progress after rejection, reopen from done, or end as cancelled. Compatibility boolean remains temporarily during migration.

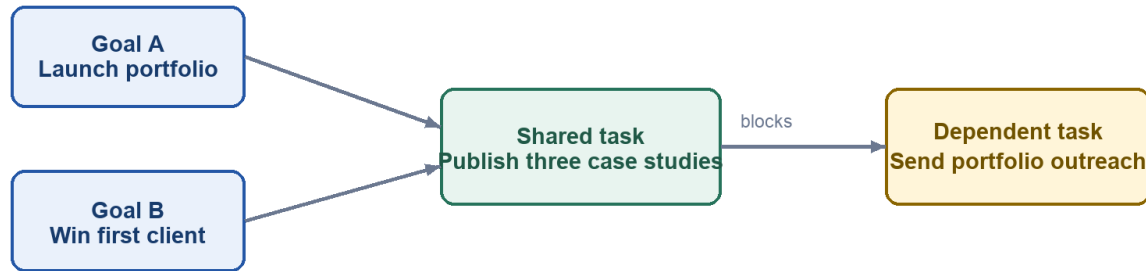
Migration strategy

1. Create personal workspace for every user.
2. Attach existing plans to personal workspace.
3. Add new task fields and GoalTask.
4. Backfill one GoalTask per existing task.
5. Dual-write legacy Task.goal and new association.
6. Compare counts, ownership, completion state, and checksums.
7. Switch reads to associations.
8. Make legacy goal reference nullable.
9. Stop dual-write after rollback window.
10. Remove legacy field in later release.

Migration rule: No big-bang migration. Every step must permit rollback and data parity checks.

10. Goal connection graph

Connected goal graph: one task, several outcomes



Completion happens once. Both goals gain progress. Removing one goal link does not delete shared task.

Graph semantics

Node types: Goal and Task. Edge types: goal contains task, goal supports or overlaps goal, goal conflicts with or duplicates goal, task blocks or enables task.

Shared work is not represented by duplicate task nodes. One canonical Task connects to several goals through GoalTask.

Graph requirements

- Connect existing task to another goal.
- Complete shared task once and update every linked goal.
- Unlink from one goal without deleting canonical task.
- Warn before editing task used by several goals.
- Fork shared task when meanings diverge.
- Reject blocking cycles.
- Derive goal dependencies from underlying tasks when useful.
- Require approval for every AI-suggested link.
- Show reason, evidence, and confidence.
- Filter graph by plan, status, owner, edge type, and timeframe.

- Explain connection paths and highlight critical blockers.

PostgreSQL adjacency tables are sufficient initially. Consider graph database only after real workloads require deep traversal or thousands of nodes per workspace.

11. Accountability and teams

Workspace roles

Owner, Manager, Member, Accountability Partner, and Viewer. Role alone does not expose every object; plan participation and task assignment remain required.

Verification policies

Policy	Default use	Completion authority
self	Personal goals	Performer may complete
peer_required	Accountability pairing	Assigned peer approves
manager_required	Team goals	Active manager approves
assigned_verifier_required	Special workflows	Named verifier approves

Completion workflow

1. Performer starts task.
2. Performer submits completion request.
3. Optional note, link, or attachment is included.
4. Assigned verifier accepts or rejects.
5. Acceptance moves task to done.
6. Rejection returns task to in_progress with reason.
7. Event log records actor, timestamp, policy, and previous state.

Privacy requirements

- Pairing requires invitation and acceptance.
- Partner sees only shared goals or tasks.
- Manager access remains inside team workspace.
- Personal plans never become visible through employment membership.

- Revoked membership loses access immediately.
- Blocking and reporting partner is supported.
- Evidence retention is configurable.
- Every verification remains auditable.

12. Calendar scheduling

Django owns calendar integration. Engine receives normalized availability, working preferences, and free/busy windows, never provider credentials.

- OAuth connection and revocation
- Timezone, working hours, focus preferences, and buffers
- Free/busy extraction with minimal event detail
- Task scheduling proposals
- User approval before event creation
- Several blocks for splittable tasks
- Idempotent event create, update, and delete
- ScheduleBlock linked to external calendar event identifier
- Conflict detection and rescheduling proposal
- Daylight-saving and timezone test coverage
- Calendar deletion never deletes task

Recommended initial provider: Google Calendar, because current authentication already uses Google OAuth. Keep provider contract generic.

13. Momentum analytics

Momentum requires event history, effort estimates, schedule blocks, verified completion, and versioned plan baselines. Building it earlier produces misleading charts.

Inputs

- Verified completed effort and planned effort
- On-time completion
- Planned sessions completed

- Blocked duration and dependency resolution
- Scope changes and reopened work
- Meaningful inactivity periods

Initial momentum model

Component	Weight	Meaning
Pace	40%	Completed effort against current plan
Schedule adherence	25%	Planned sessions and deadlines met
Consistency	20%	Sustained execution across active days
Blocker recovery	15%	Blocked work resolved without long stagnation

Missing component redistributes weight. Formula is versioned as momentum_v1. Every score exposes underlying values.

Views

- Momentum line per goal and overall workspace trend
- Planned versus completed effort
- Completion burn-up
- Blocked-task duration
- Scope-change markers
- On-track, at-risk, and stalled state
- Explanation panel naming why momentum changed

Compare users against their own plans and previous pace. Do not rank employees or accountability partners. New plan revisions create new baselines without rewriting history.

14. API and event surface

Indicative Django endpoints

- POST /api/v1/decomposition-runs/
- GET /api/v1/decomposition-runs/{id}/
- POST /api/v1/decomposition-runs/{id}/answer/
- POST /api/v1/decomposition-runs/{id}/accept/

- GET /api/v1/plans/{id}/graph/
- POST /api/v1/tasks/{id}/goal-links/
- POST /api/v1/task-dependencies/
- POST /api/v1/relationship-suggestions/{id}/accept/
- POST /api/v1/tasks/{id}/start/
- POST /api/v1/tasks/{id}/request-completion/
- POST /api/v1/completion-reviews/{id}/approve/
- POST /api/v1/completion-reviews/{id}/reject/
- POST /api/v1/workspaces/{id}/invitations/
- GET /api/v1/momentum/?range=30d
- POST /api/v1/calendar/connections/
- POST /api/v1/schedule-proposals/{id}/accept/

Every mutation receives idempotency key and expected resource version.

Core activity events

- plan.created, plan.revised, plan.restored
- goal.created, goal.updated, goal.completed
- task.created, task.started, task.blocked, task.unblocked
- task.completion_requested, task.verified, task.rejected, task.reopened
- task.goal_linked, task.goal_unlinked, dependency.added, dependency.removed
- schedule.proposed, schedule.accepted, schedule.completed, schedule.missed
- membership.invited, membership.accepted, membership.revoked

15. Security and reliability

- Short-lived service credentials or mutual TLS between Django and engine.
- Engine never receives user JWT or calendar OAuth token.
- Object-level workspace and plan authorization on every query.
- Context allowlist, redaction, freshness, and provenance.
- Treat messages and calendar text as untrusted data.

- Engine returns proposals only; no arbitrary tool execution.
- Django revalidates every engine result.
- Encrypt provider and calendar secrets.
- Disable raw prompt logging by default.
- Set cost limits per user, workspace, run, and provider.
- Use circuit breakers and distributed tracing.
- Support data export, deletion, and retention policy.
- Keep audit logs append-only from application perspective.

16. Delivery plan

Assumption: small cross-functional team. Ranges represent engineer-weeks, not calendar promises.

Phase 0 - Contracts and measurement | 1-2 weeks

- Approve PRD and architecture decisions.
- Define canonical task, run, graph, and event schemas.
- Create authorization matrix and 100-200 prompt golden set.
- Record quality, latency, edit, and cost baselines.
- Correct documentation drift.

Exit: Versioned contracts and measurable baseline.

Phase 1 - Domain foundation | 3-4 weeks

- Workspace and membership models.
- Personal workspace backfill.
- Canonical Task plus GoalTask.
- Task state machine, activity events, and audit trail.
- Compatibility APIs and migration verification.

Exit: Current product runs on new domain model with no data loss.

Phase 2 - Engine extraction | 4-6 weeks

- Go service skeleton and Gemini adapter.
- Second provider adapter.
- Run and attempt records.
- Actionability validators, targeted repair, fallback.
- Context snapshot service and shadow rollout.

Exit: At least 95% calendar-ready tasks on evaluation set; malformed output never persists.

Phase 3 - Scheduling | 3-4 weeks

- Duration and scheduling constraints.
- Availability context and calendar connection.
- Schedule proposal review and idempotent synchronization.

Exit: Accepted plan produces approved calendar blocks.

Phase 4 - Connection graph | 3-5 weeks

- Shared task links, dependencies, relationships.
- AI suggestions and review.
- Graph projection API and UI.
- Cycle and cross-workspace protection.

Exit: One task advances several goals; blockers visible.

Phase 5 - Accountability and teams | 4-6 weeks

- Invitations, memberships, assignments, policies.
- Completion review, notifications, privacy controls.
- Manager and team views.

Exit: Unauthorized user cannot verify or view work; every review auditable.

Phase 6 - Momentum | 3-4 weeks

- Event aggregation and baseline-aware calculations.
- Daily and near-real-time snapshots.
- Goal and workspace graphs with explanations.

Exit: Every displayed metric reproducible from events.

Phase 7 - Beta hardening | 2-3 weeks

- Load, chaos, and provider-failure testing.
- Security, accessibility, retention, and mobile review.
- Gradual production rollout.

Exit: Beta-ready platform with rollback and incident controls.

Expected total: roughly 20-30 engineer-weeks. Small team: about five to eight months.

17. Acceptance criteria

Engine

- Same idempotency key never creates duplicate plan.
- Invalid or incomplete output never reaches domain tables.
- Provider failure follows bounded retry and fallback policy.
- Vague work receives targeted repair or clarification.
- Each accepted task includes completion criteria and effort estimate.
- Provider swap does not require domain rewrite.

Graph

- One shared task appears once and completion updates every linked goal.
- Blocking cycles are rejected.
- Unlinking does not delete canonical task.

- AI relationship suggestions require approval.

Accountability

- Performer cannot self-complete under peer or manager policy.
- Only active authorized verifier can approve.
- Every review and reopen event is recorded.
- Personal plans remain hidden from team managers.

Momentum

- Same event sequence always produces same score.
- Scope changes remain visible in history.
- Score explanation names contributing factors.
- Metric never uses inaccessible cross-user data.

18. Success metrics

Class	Metric
Product	Decomposition acceptance without major edits
Product	First calendar-ready task scheduled within 24 hours
Product	Weekly verified-task completion
Product	Shared tasks reused across goals
Product	Median blocker resolution and review turnaround
Product	Four-week active-plan retention and stalled-goal recovery
Technical	Candidate-ready p95 below 30 seconds
Technical	API mutation p95 below 400 ms
Technical	Valid candidate rate above 99%; actionability above 95%
Technical	Analytics freshness below five minutes
Technical	Duplicate mutation and cross-workspace exposure: zero
Technical	LLM cost per accepted plan tracked and bounded

19. Risks and mitigations

Risk	Mitigation
Microservice slows delivery	Shadow extraction, feature flags, no big-bang rewrite
AI merges merely similar tasks	Suggestions only; explicit confirmation
Shared edits cause surprise	Impact preview and fork option
Retries inflate cost	Targeted repair, budgets, circuit breakers
Manager verification becomes surveillance	Consent, workspace limits, audit, no ranking
Momentum gets gamed	Effort-aware metrics, event history, own-baseline comparison
Old plans lack history	Mark metrics available since migration date
Calendar creates duplicates	Stable external IDs and idempotency
Provider semantics differ	Conformance suite and golden evaluations

20. Decisions to lock

Decision	Recommended default
Service language	Go target after two-week conformance spike; keep Python fallback
Security boundary	Workspace model, including personal workspace for every user
Shared work	Canonical Task with GoalTask associations
Verification	Configurable self, peer, manager, or named verifier policy
Graph storage	PostgreSQL adjacency tables first
Analytics source	Immutable ActivityEvent ledger
Calendar authority	Django owns OAuth and external writes

Appendix A - Permission matrix

Role	Default scope	Allowed execution actions
Personal owner	Owned personal plans	Edit, execute, request, verify when policy=self
Team member	Assigned team tasks	Execute and request; cannot approve manager_required
Manager	Participating team plans	Assign, review, approve manager_required

Role	Default scope	Allowed execution actions
Accountability partner	Explicitly shared tasks	View and approve peer_required tasks only
Viewer	Explicitly shared plans	Read only
Workspace owner	Workspace administration	Manage membership; no automatic personal-plan access

Appendix B - Planning Foundation Sprint

Objective: remove architectural ambiguity before feature implementation and create evidence for Go extraction.

Workstream	Scope	Deliverable
ADR-001	Django-engine boundary and ownership	Approved architecture decision
ADR-002	Canonical Task and GoalTask migration	Migration and rollback plan
Contract	Request, result, context, quality schemas	Versioned JSON Schema package
Evaluation	100-200 representative prompts	Golden dataset and scorecard
Baseline	Current Gemini workflow	Quality, latency, edit, and cost report
Go spike	Gemini and OpenAI adapter conformance	One end-to-end run with tracing
Gate	Review findings	Go, Python service, or deferred extraction decision

Go acceptance gate

- Both provider adapters produce canonical schema.
- Cancellation, timeout, retry classification, and usage reporting work.
- OpenTelemetry trace spans cross Django, engine, and provider call.
- Evaluation quality matches or exceeds current Python flow.
- Operational complexity remains acceptable for current team.
- Extraction does not block domain foundation delivery.

Appendix C - Source notes

Codebase snapshot reviewed:

- Backend: goals/models.py, goals/services.py, workflow/ai_engine.py, conversations/models.py, conversations/services.py, conversations/views.py, tasks/models.py
- Frontend: src/lib/api/types.ts, src/lib/api/plans.ts, src/lib/api/chat.ts, src/components/console/*, src/app/(dashboard)/console/[id]/page.tsx
- Validation: Django test suite and frontend ESLint

External technical references:

[OpenAI official Go SDK](#)

[Google GenAI SDK libraries](#)

[Gemini structured outputs](#)

[OpenTelemetry Go](#)

Final priority: First release should be Trustworthy Planning: canonical tasks, activity events, extracted engine, actionability repair, and schedule proposals. Graph follows. Accountability follows workspace and graph foundations. Momentum comes last because trustworthy analytics require trustworthy execution data.